



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



Escola Tècnica
Superior d'Enginyeria
Informàtica

Escola Tècnica Superior d'Enginyeria Informàtica
Universitat Politècnica de València

BollyKO's

Grado en Informática Industrial y Robótica

Autor: Carla Castellanos Gómez

Pablo Gómez San Frutos

Gabriel Arnedo Espinosa

Nuria Urrecho Torres

Grupo: PR2-A05

Fecha: 06/06/2025

Contacto: garnesp@etsinf.upv.es

Índice

1. Resumen	4
2. Introducción	5
3. Propuesta de automatización	6
Análisis de alternativas	6
Objetivos del proyecto	6
Objetivos específicos	6
Funcionamiento tras automatización	6
4. Diseño de la solución	8
Modulo esp32	8
Base de datos	10
Distribución del trabajo	13
Diagrama de comunicaciones	14
5. Desarrollo e Implementación	16
5.1. Gestión del proyecto	16
5.2. Pruebas y Validación	17
5.3. Costes y Beneficios	18
6. Normativa y Regulación. Seguridad	21
Seguridad laboral	21
Regulación técnica y marcado CE	21
Principales normativas y estándares aplicables	21
Aplicación práctica de la normativa en BollyKO's	22
Conclusión	22
7. Desarrollo de Software y Algoritmos	23
8. Conclusiones y recomendaciones	32
9. Referencias	33
Normativa consultada para la sección 6	33
10. Anexos	34
10.1. Manual de Instalación y Funcionamiento	34
10.2. Manual del programador	34
10.3. Otra documentación adicional	34

1. Resumen

Esta propuesta de automatización nace con la intención de mejorar un proceso manual: una estación de llenado y empaquetado de bollos industriales. No solo hemos automatizado dicho proceso, sino que además hemos añadido funcionalidades que mejoran la calidad del servicio y la eficiencia de producción

El objetivo principal del proyecto es optimizar esta estación, aumentar la productividad y mejorar indicadores clave de rendimiento(KPI's), lo que se traduce directamente en un incremento del rendimiento global de la fábrica e, inevitablemente, en mayores ingresos. Aunque el análisis económico se detalla en su correspondiente apartado, ya podemos adelantar que se han obtenido indicadores muy prometedores, cumpliendo todos y cada uno de los KPI's definidos.

Entre las dificultades previstas destacamos el reto de introducirnos en el sector alimentario, que puede presentar limitaciones por falta de conocimientos técnicos específicos. También hemos tenido que afrontar plazos ajustados que han supuesto una carga importante de trabajo. Además, el coste final del sistema podría ser elevado para pequeñas empresas, a pesar de ser uno de nuestros públicos objetivos.

Para concluir, damos las gracias a todos los profesores de la asignatura de PR2 por guiarnos en este proyecto, a los compañeros de otros grupos que nos han ayudado cuando teníamos alguna dificultad y, en especial, a Jose Luis, a Charo y a Natalia, sin los cuales este proyecto no se podría haber realizado.

2. Introducción

En el sector de la alimentación industrial, la eficiencia y la calidad son factores clave para mantener la competitividad. Este proyecto se sitúa en el contexto de una pequeña planta productora de bollos que opera con procesos mayoritariamente manuales en su estación de empaquetado y llenado. A pesar de ofrecer un producto de calidad, la empresa enfrenta limitaciones en capacidad de producción, tiempos de respuesta y trazabilidad del proceso.

Nuestro grupo de trabajo ha asumido el rol de consultores externos especializados en automatización industrial. A partir del proceso actual, hemos detectado oportunidades de mejora que justifican la introducción de una solución automatizada.

La automatización propuesta no solo busca sustituir tareas repetitivas y susceptibles de error humano, sino también mejorar el control sobre el proceso, asegurar una producción más constante, facilitar la integración con sistemas de supervisión y gestión de datos. Además se incorporan tecnologías avanzadas como la robótica colaborativa, sistemas embebidos, visión artificial y comunicaciones mediante protocolos como MQTT.

El proceso a automatizar es la estación de llenado y empaquetado de bollos. Esta estación actualmente se encarga de recibir las masas, inyectar el relleno y colocar los envoltorios en cajas para su distribución. El proyecto contempla la implementación de una estación robotizada capaz de realizar estas tareas de forma sincronizada, eficiente y segura.

3. Propuesta de automatización

Análisis de alternativas

En principio se valoraron varias opciones. Obviamente se descartó seguir con el proceso manual debido a su poco rendimiento y escalabilidad. Se valoró desde una mejora parcial basada en sensores y actuadores simples para ayudar al proceso y la sustitución total por maquinaria especializada. Estas opciones fueron descartadas en pos de la automatización con cobots, priorizando así una respuesta intermedia aumentando la flexibilidad escalabilidad y coste moderado.

Esta opción permite mantener la esencia del proceso pero con una mejora sustancial en la eficiencia, trazabilidad y control del sistema

Objetivos del proyecto

Objetivos específicos

En cuanto a los objetivos específicos, hemos definido varios para poder saber si el proyecto ha cumplido las metas:

- **Automatizar** completamente el proceso mediante una estación con tecnologías integradas
- **Incorporar tecnologías** que permiten una trazabilidad completa del producto
- **Reducir** el número de **errores** y tiempos muertos en la línea de producción
- Aumentar la **productividad** sin aumentar los costes del personal

Funcionamiento tras automatización

Las bandejas de bollos entran por una cinta a la estación, donde una cámara las visiona y procesa si son o no defectuosas. En caso de que lo sean, un pistón actuaría para sacar la bandeja de la producción. Una vez pasado el control, pasamos a una sección de la cinta donde el primer robot UR16e, equipada con una herramienta de inyección triple (que hemos diseñado nosotros) que rellena los bollos. Una vez rellenos, pasan a una máquina donde se embolsan y salen por otra cinta.

En esta se mueve hasta que un sensor detecta las bolsas y, otro robot UR16e con una herramienta de ventosa recogería las bolsas y las pondría en una caja(tres bolsas por cada caja).

Una vez con tres bolsas en la caja, la cinta se pondría a funcionar y se llevaría la caja a otra sección de la fábrica y alimentaría la caja con otra cinta. Todo esto siempre vigilado con un sistema de parada de emergencia que para la fábrica al instante en caso de necesidad.

Además hemos implementado un sistema de pedidos online por una página web, donde puedes ver cómo se genera, rellena y sale el pedido con tu nombre. Además hemos construido una cinta desde cero, completamente funcional que dispensa tantos dulces como bollos hayas pedido cuando acaba la simulación. Esto tenía el fin de mejorar la visibilidad en la feria de proyectos y se podría usar para enseñarle al cliente una demo con una sección física para que vea el funcionamiento, la escalabilidad y la personalización que va a tener sobre su fábrica.

Todo esto controlado por una ESP32-S3 que lo gestiona todo comunicándose con sensores, actuadores, bases de datos y supervisión a través de MQTT.

KPIs definidos

Estos son los objetivos medibles que nos establecimos:

- Tiempo de empaquetado
- Tiempo de relleno
- Tiempo de detección de errores
- Estandarización del peso del producto
- Bollos por minuto

Otros aspectos afectados

Si bien se requieren nuevos materiales y la compra de nuevo equipamiento, la reducción del trabajo manual al personal es considerable. Además hemos presupuestado una partida de dinero reservada a formación al profesional para que puedan seguir trabajando para la empresa revisando el funcionamiento de la estación. Además va a facilitar acceder a los datos de producción, va a ser fácilmente escalable y podremos planificar de forma precisa futuras ampliaciones.

Impactos adicionales

- **Ambientales:** al reducir los errores y evitar el relleno de más chocolate del necesario, reducimos el impacto de las basuras en el medio ambiente, si bien es cierto que aumentaremos nuestro consumo eléctrico de forma considerable
- **Tecnológicos:** al introducir tecnologías de IoT, visión artificial y sistemas embebidos nos adaptamos al siglo XXI. Incluso podríamos acceder a subvenciones gubernamentales.
- **Logísticos:** mayor previsibilidad y sincronización con sistemas de almacenamiento y envío.
- **Salariales:** reducir salarios al no necesitar tantos trabajadores o al reducirles la jornada.

Límites y restricciones

Partimos de la premisa de que había que ajustar el presupuesto para que fuese atractivo a PYMES sin tantos ingresos. Además el proyecto se ha desarrollado en paralelo a otras actividades académicas, por tanto el tiempo ha sido limitado. Además carecemos de ciertos conocimientos técnicos que hemos intentado suplir mediante consulta y asesoramiento.

4. Diseño de la solución

MODULO ESP32

El sistema embebido desarrollado está basado en una ESP32-S3, utilizada como nodo central de control de actuadores físicos, recepción de eventos, lectura de códigos QR y coordinación con otros componentes del sistema, como el robot simulado en RoboDK y una base de datos alojada en [Render.com](https://render.com).

Componentes gestionados por la ESP32

Los dispositivos conectados a la ESP32 permiten realizar tareas físicas del sistema de producción, detección de defectos y control general:

- **Servomotores reales:** tres servomotores controlan la dispensación de caramelos según el sabor (chocolate, fresa y vainilla).
- **Pulsador físico:** botón de parada de emergencia conectado a una interrupción por hardware.
- **Cinta transportadora:** se activa al inicio de la dispensación.
- **LED auxiliar:** utilizado como indicador visual durante el evento de QR detectado.
- **Cámara integrada:** permite la lectura continua de códigos QR para trazabilidad y control de calidad.

Lógica y tareas del sistema

La funcionalidad de la ESP32 se organiza mediante tareas concurrentes bajo FreeRTOS, gestionadas por el sistema operativo integrado en la placa. Las principales tareas son:

- **Task_ProcesarTareas:** recibe pedidos en formato JSON vía MQTT, y los introduce en una cola circular FIFO protegida por mutex, para ser procesados de forma secuencial.
- **TareaControlDispensador:** espera la señal “fin_produccion” (proveniente del sistema de simulación RoboDK). Al recibirla, extrae el siguiente pedido de la cola FIFO y activa el servo correspondiente al sabor tantas veces como cantidad de bollos tenga el pedido.
- **TareaPulsador:** Utiliza una interrupción por hardware para detectar una pulsación física. La ISR simplemente marca una variable volatile, y la tarea responde enviando un evento “stop” o “start” vía MQTT, lo que permite simular la parada de emergencia.

- **QRCodeReader:** se ejecuta en un segundo núcleo de la ESP32. Captura imágenes mediante la cámara integrada, procesa códigos QR usando la librería `quirc`, y:
 - Si el QR es válido, envía su contenido (`bollo_defectuoso`) como evento MQTT al sistema y genera un POST HTTPS hacia la base de datos externa para registrar el bollo defectuoso.
 - Si el QR está dañado o no se puede leer, no ocurre nada.

Comunicación externa e interna

- Internamente

La comunicación entre tareas se realiza mediante:

- Cola FIFO (`xQueue`): para los pedidos
- Estructura `ColaPedidos`: encapsula el acceso a buffers y evita el uso de variables globales.
- Semáforos binarios: para sincronización por evento (como `fin_produccion`).
- Mutex (`xSemaphoreCreateMutex`): para proteger acceso a datos compartidos.

- Externamente

El módulo se comunica con el resto del sistema mediante dos protocolos:

1. MQTT: para eventos en tiempo real como:

- “nuevo_pedido”
- “fin_produccion”
- “stop” / “start”
- “bollo_defectuoso”

Se emplean los topics

- `richi5/giirrob/esp32/recibir`
 - `{evento:"nuevo_pedido", nombre:"Juan Perez", sabor:"chocolate", cantidad: 2}` (lo recibimos desde la web)
 - `{evento:"fin_produccion"}` (lo recibimos desde RoboDK)
- `richi5/giirrob/esp32/enviar`
 - `{evento:"nuevo_pedido", nombre:"juan_perez1", sabor:"chocolate", cantidad: 2}` (lo enviamos a RoboDK)
 - `{evento:"start/stop"}` (lo enviamos a RoboDK)
 - `{evento:"bollo_defectuoso"}` (lo enviamos a RoboDK)

2. HTTPS (HTTPClient): para registrar eventos de producción defectuosa

Se envía una solicitud POST al endpoint:

<https://richibollos.onrender.com/api/defectuoso>

Con cuerpo JSON que contiene el hecho_id y el motivo del fallo.

Comportamiento general

1. Un pedido llega a la ESP32 por MQTT → Se guarda en la cola y se envía a RoboDK
2. El sistema simulado de RoboDK notifica el “fin_produccion” → la ESP32 activa el dispensador.
3. Si el usuario pulsa el botón físico → se envía “stop” o “start” por MQTT
4. En paralelo, la cámara escanea continuamente códigos QR:
 - Si hay éxito → se notifica vía MQTT y se actualiza la base de datos vía HTTPS.
 - Si hay fallo → no pasa nada.

BASE DE DATOS

Diseño de Persistencia de información

El sistema utiliza una base de datos relacional PostgreSQL, compuesta por tres tablas fundamentales (pedidos, hechos y defectuosos) para la implementación de nuestro trabajo. Está pensada y hecha, de manera que solo hemos puesto aquella información que era necesaria guardar y que necesitábamos, haciendo así más sencillo todo el proceso de comunicación.

Se ha optado por este modelo por:

- Integridad de los datos, dado que permite definir restricciones como claves primarias.
- Compatibilidad dado que se integra fácilmente con el flask de python y compatible con el servidor utilizado (Render).
- Facilidad de acceso y consulta de manera sencilla y segura.

Modelo Lógico

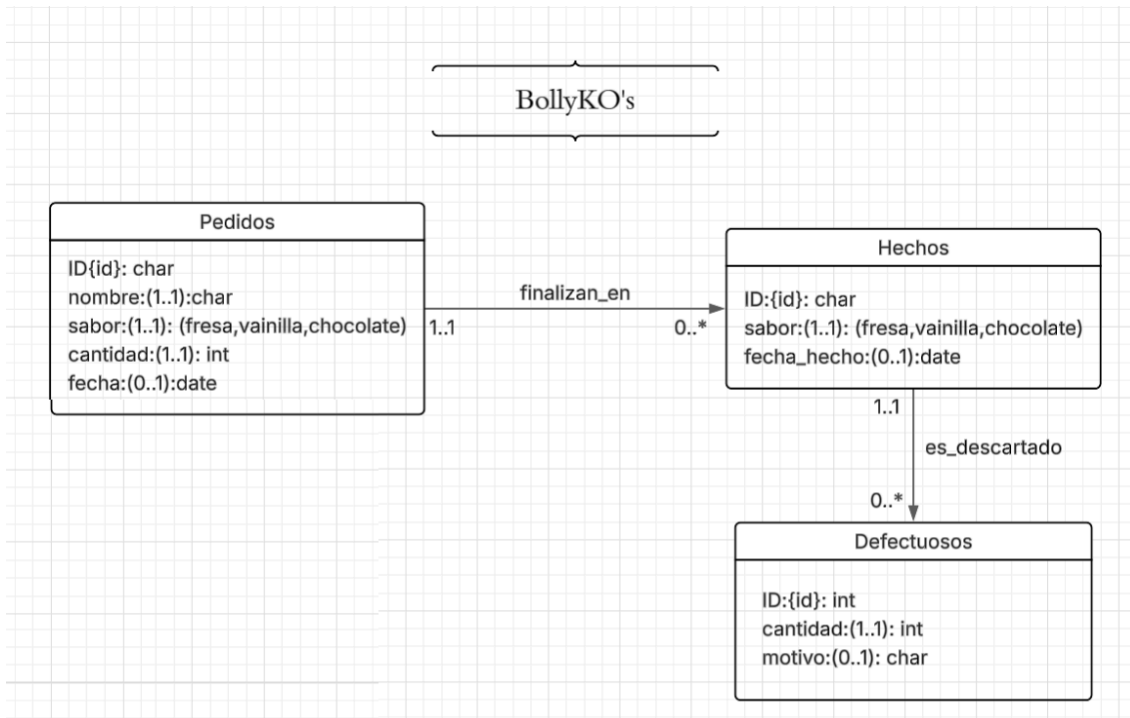
El modelo se estructura con tres entidades principales:



- Pedidos: Representa cada solicitud hecha por un cliente desde la web. Incluye el nombre, el sabor elegido y la cantidad.
- Hechos: Mantiene un contador de bollos correctamente, nos permite tener más control y diferenciarlos por sabor.
- Defectuoso: Almacena cuantos bollos han sido defectuosos.

La tabla de pedidos y hechos, se gestiona a través del formulario de la web, donde se registran los datos. La tabla de defectuosos, es gestionada por la ESP32 con la lectura de un código QR (se puede encontrar dentro de la web)

En este diseño conceptual, dónde se pueden observar mejor las conexiones.



Modelo Relacional

Este es el modelo relacional de nuestra base de datos, que una vez pasado al lenguaje SQL, la hemos podido implementar.

```
PEDIDOS(id,nombre, sabor, cantidad, fecha)
```

```
CP: {id}
```

```
VNN: {sabor, cantidad, nombre}
```

```
HECHOS(id, pedido_id, sabor, fecha_hecho)
```

```
CP: {id}
```

```
VNN: {pedido_id, sabor}
```

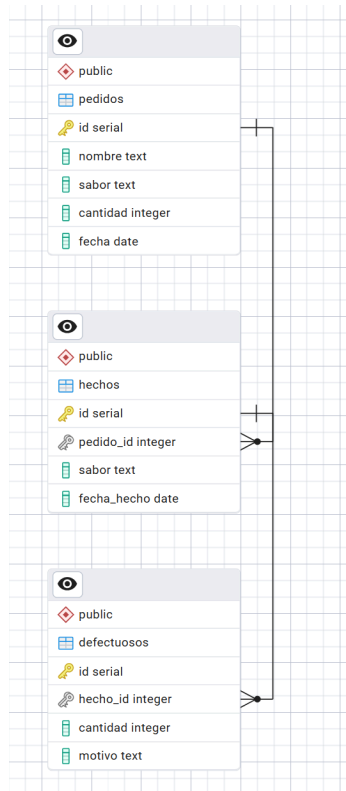
```
CAj: {pedido_id} → PEDIDOS(id)
```

```
DEFECTUOSOS(id, hecho_id, cantidad, motivo)
```

```
CP: {id}
```

```
VNN: {hecho_id, cantidad}
```

```
CAj: {hecho_id} → HECHOS(id)
```

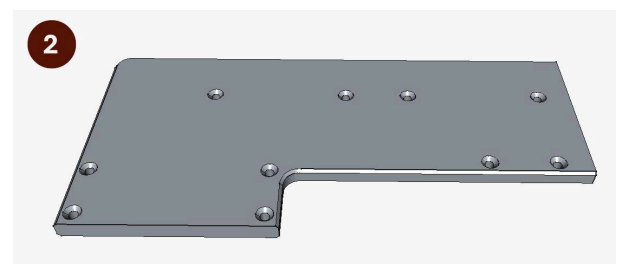
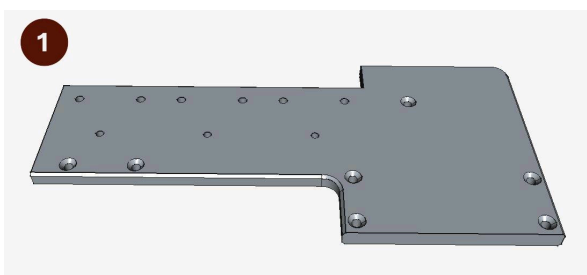


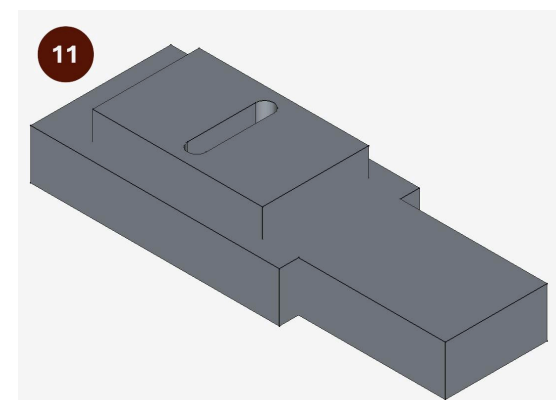
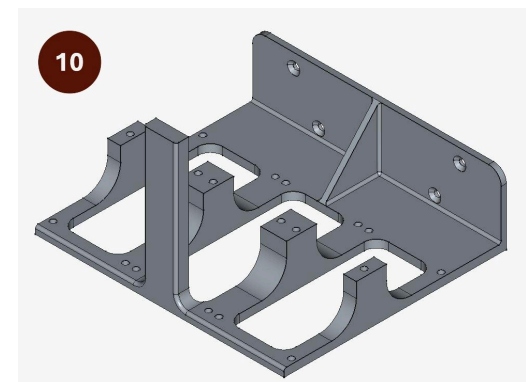
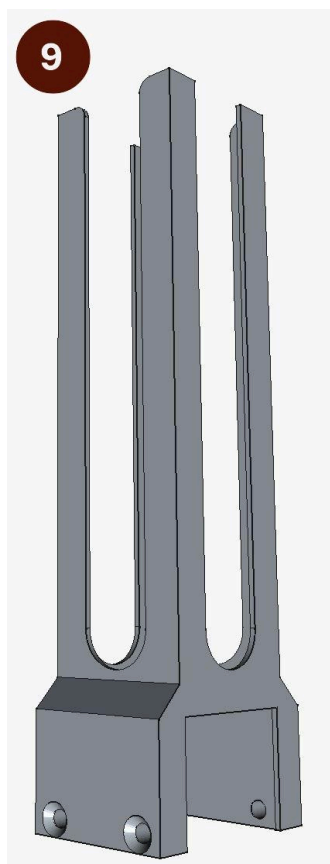
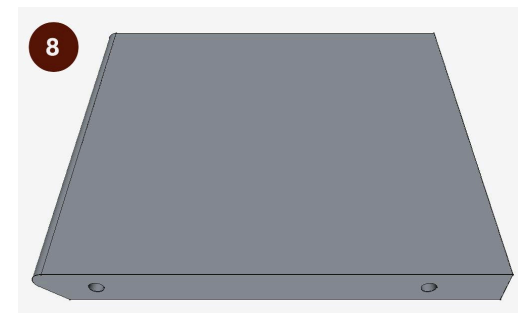
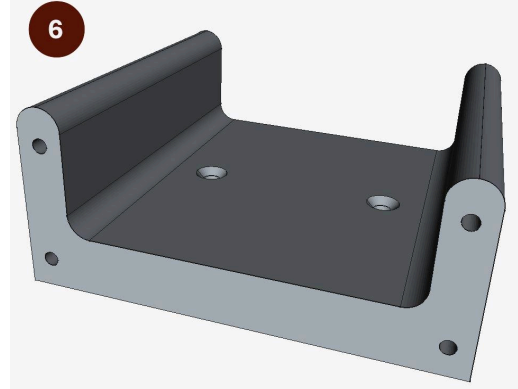
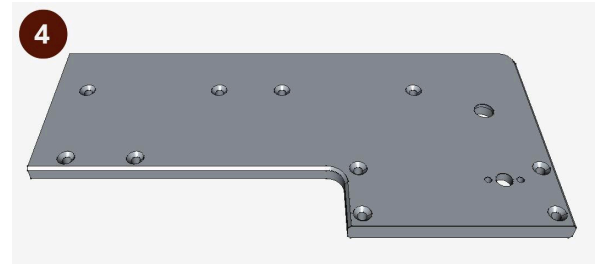
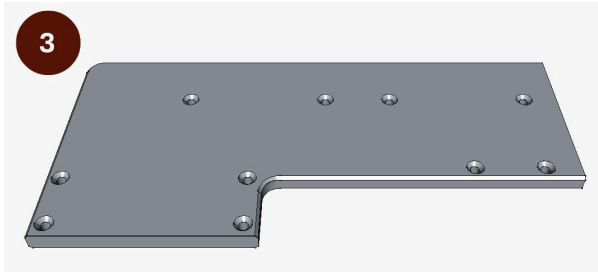
CINTA

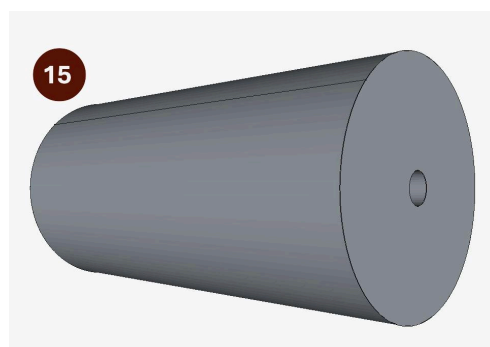
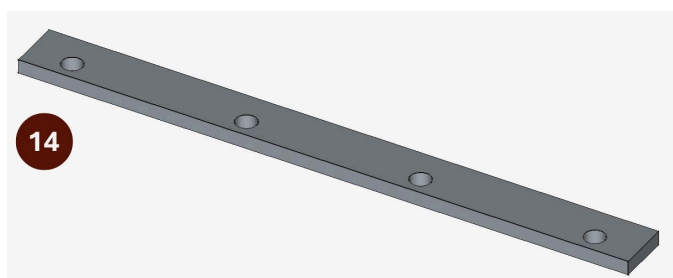
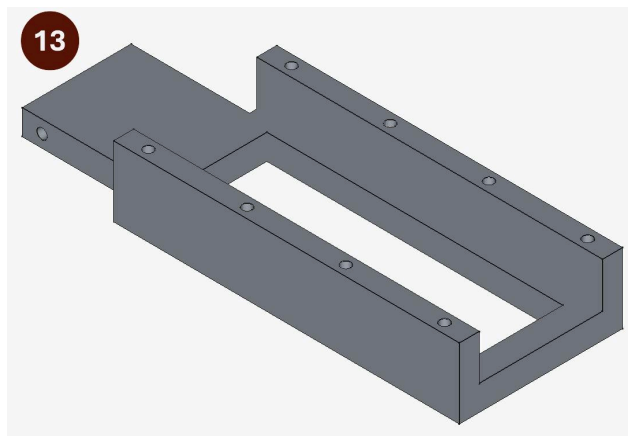
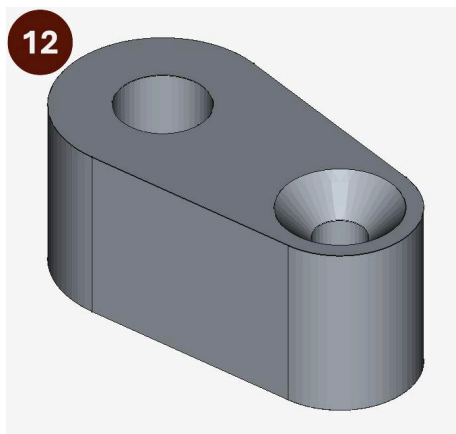
Para poder implementar los requisitos de la instrumentación sobre la ESP y, sobre todo, de cara al público de la feria, se ha implementado un prototipo de maqueta de una cinta transportadora. La maqueta simulará el funcionamiento de la cinta que suministra las bolsas de bollos personalizados, de forma que en función del pedido realizado en la web (1, 2 o 3 bollos de fresa, vainilla o chocolate), la cinta se pondrá en marcha y suministrará el pedido.

Para el montaje, la estructura de la se ha realizado mediante una impresora Wanhao Duplicator 6 con filamento blanco PLA de 1.75mm de grosor. Los diseños de cada pieza han sido realizados en el programa FreeCad y se han exportado en formato .stl.

A continuación se muestran los diseños de cada pieza:







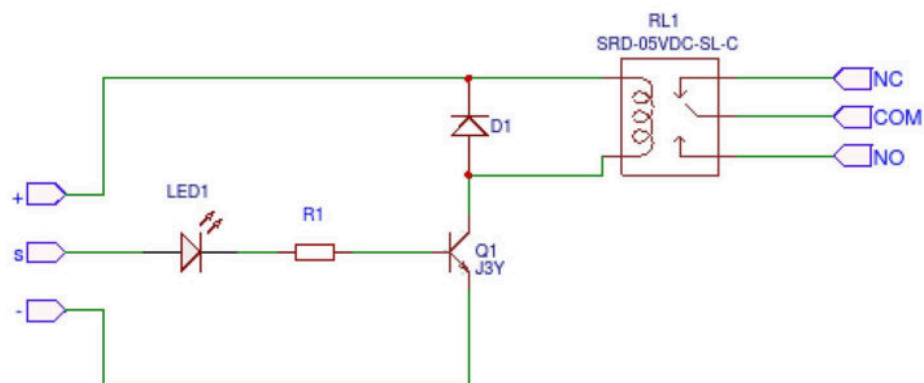
- **FIGURA 1:** panel interior izquierdo (donde irán los suministradores)
- **FIGURA 2:** panel interior derecho
- **FIGURA 3:** panel exterior izquierdo
- **FIGURA 4:** panel exterior derecho (donde irán las poleas y el motor)
- **FIGURA 5:** pletina para asegurar la unión entre los paneles
- **FIGURA 6** (2uds.): base para las patas de la cinta
- **FIGURA 7:** base central de apoyo para la tela de la cinta
- **FIGURA 8** (2 uds.): bases izquierda y derecha para el apoyo de la tela de la cinta
- **FIGURA 9** (3 uds.): contenedor de los caramelos
- **FIGURA 10:** base para la colocación de los contenedores, servomotores, correderas, etc.
- **FIGURA 11** (3 uds.): corredera para empujar los caramelos
- **FIGURA 12** (3 uds.): leva para que el servomotor mueva la corredera
- **FIGURA 13** (3 uds.): guía para que la corredera se mueva dentro de ella
- **FIGURA 14** (6 uds.): pletina para atornillar la guía correctamente
- **FIGURA 15** (2 uds.): rodillos para la rotación de la cinta

Con todo esto, la cinta resultante quedaría así:



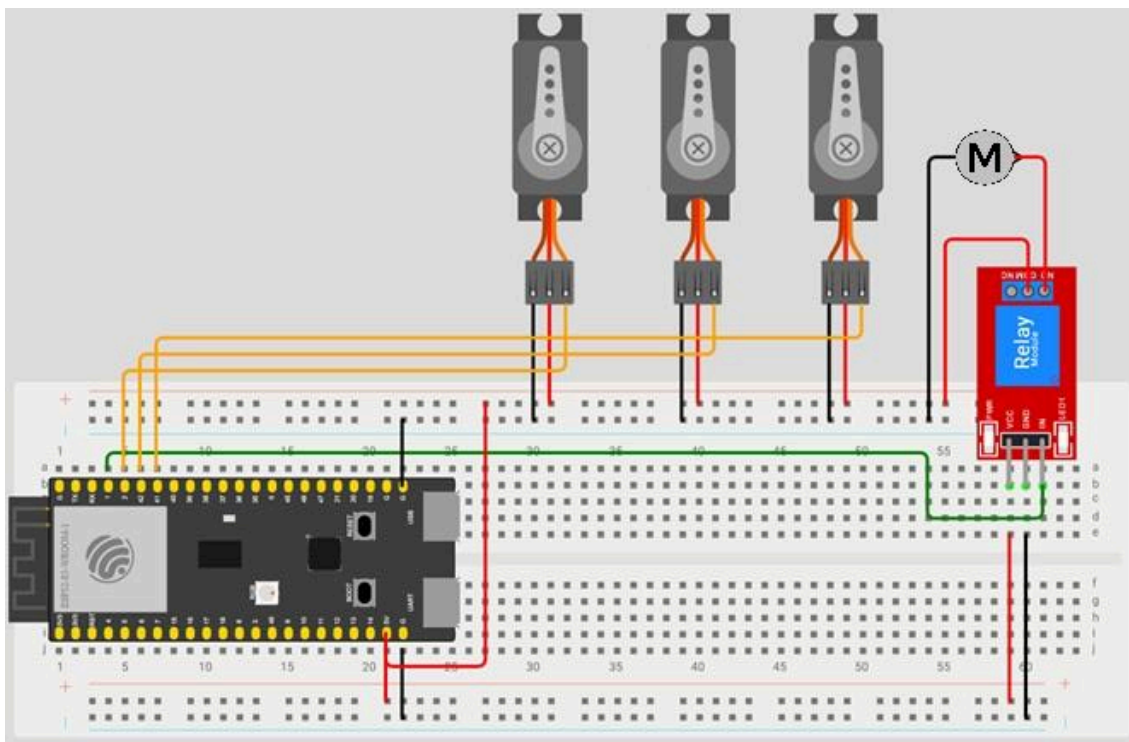
Una vez realizada la maqueta, pasamos a los componentes que se usan para hacer que funcione correctamente:

- **Motorreductor 100rpm, 6V, consumo 10mA:** aplicado para que la cinta pueda funcionar a una velocidad adecuada (con un motor normal giraría a demasiada velocidad)
- **3 servomotores MG90S:** para mover las correderas que empujarán los caramelos
- **Módulo Keyes SR1y:** módulo para hacer que la cinta se mueva solamente cuando sea necesario suministrar un pedido. A continuación se adjunta un esquema de sus componentes y su circuito:



- **2 poleas eje 3mm, 20 dientes:** para transmitir el movimiento del motor al rodillo de la cinta y hacer que ruede a la misma velocidad
- **Correa de distribución 6mm x 96mm:** para transmitir ese movimiento de una polea a otra
- **Eje de 3mm x 77mm:** eje del rodillo derecho, que es movido por el motor
- **Eje de 3mm x 60mm:** eje del rodillo izquierdo
- **Rodamiento de bolas de eje 3mm, ancho 4mm, diámetro exterior 8mm:** para asegurar que los rodillos giren fluidamente
- **4 arandelas de eje 3mm:** para que el rodamiento no gire directamente contra el rodillo y la pared
- **Cinta** de tela negra de 5cm de ancho

Con todos estos elementos, la conexión del motorreductor, los servomotores y el relé ha sido la siguiente:



De esta forma, la comunicación y el funcionamiento de la cinta consiste en una constante comunicación con la ESP.

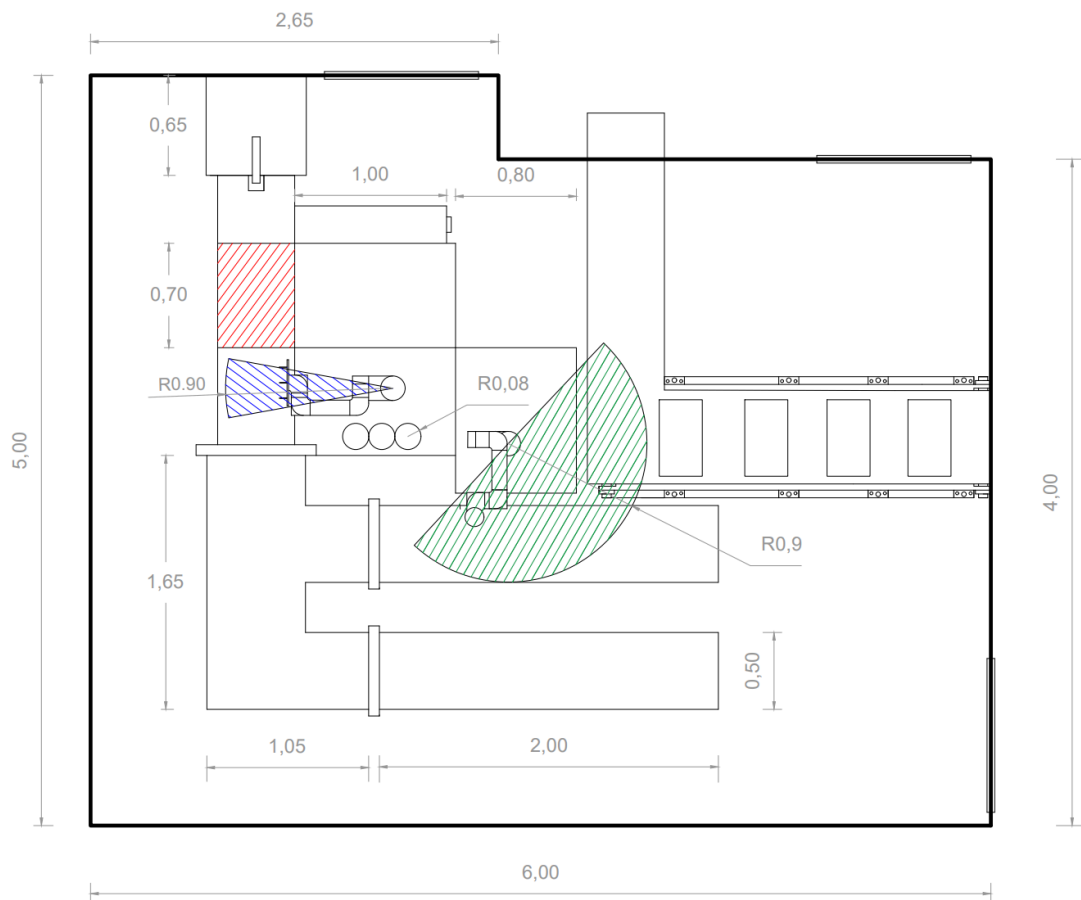
Primeramente, un usuario realizará un pedido por la web, en el cual se especificará el nombre del usuario, la cantidad de bollos que quiere (entre 1 y 3) y, por último, el sabor que desea (fresa, vainilla o chocolate). Este pedido se procesará a través de MQTT y se verá reflejado en la simulación de RoboDK.

Una vez hayan sido envasados los bollos personalizados, le llegará a la cinta un MQTT que activará el relé y que, por tanto, activará el motor durante unos segundos. Es entonces cuando, en función del pedido que se haya realizado, se moverá el servomotor

correspondiente al sabor elegido y suministrará los caramelos solicitados. El funcionamiento consiste en mover el servomotor 180° para empujar el caramelo y volver a la posición 0° tantas veces como caramelos se haya solicitado.

Finalmente, el relé se volverá a poner a 0, se parará el motor y estará de nuevo listo para afrontar otros pedidos nuevos o que se encuentren en la cola circular (implementada para poder acumular los pedidos y que se suministren por orden de llegada).

DISTRIBUCIÓN DEL TRABAJO

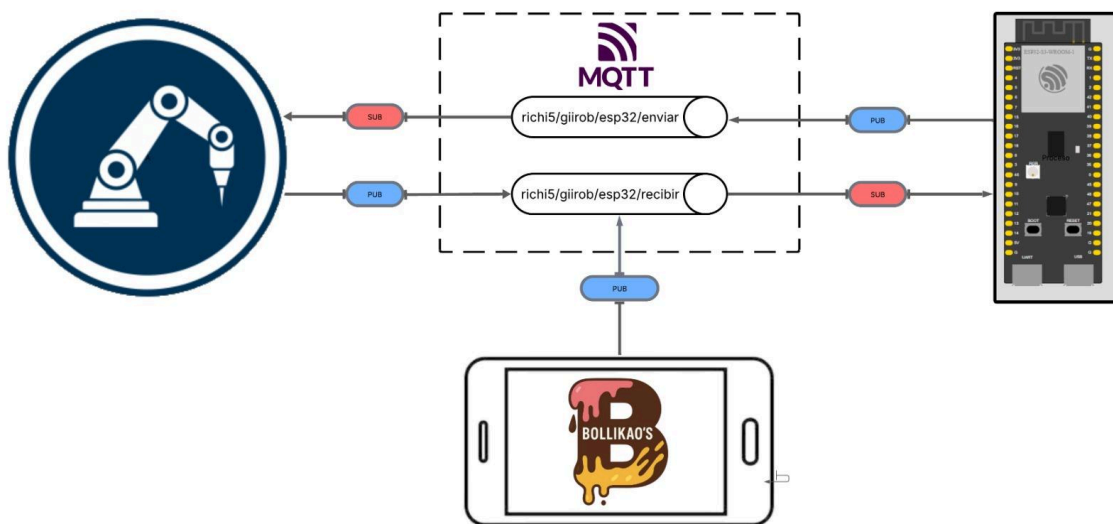


La planta está formada por:

- Cinco cintas
 - La primera para el rellenado de bollos, donde estará un robot Ur16e para rellenarlos (rango azul)
 - Dos cintas para el embolsado de bollos,
 - Una para la producción en masa, donde otro robot Ur16e lo moverá a una caja para así poder empaquetarlas
 - La otra para los personalizados, hecho adrede para que en la feria se vea más claro el pedido de cada cliente
 - Una cinta para las cajas vacías

- Y la última cinta para el empaquetado de los bollos, donde el segundo robot (rango verde) dejará cada bollo
- Dos robots
 - Uno para rellenado (rango azul)
 - Otro para empaquetado (rango verde)
- Un pistón (rango rojo) para desechar los bollos defectuosos.
- 5 sensores colocados en las distintas cintas para detectar los bollos
 - En la primera cinta hay dos
 - Uno para el pistón y detectar el defectuoso
 - El otro para el rellenado de bollos
 - En las otras cintas hay uno en cada una
 - Uno para detectar el embolsado
 - El segundo para las cajas Vacías
 - El último para las cajas una vez estén llenas

DIAGRAMA DE COMUNICACIONES



1. El cliente rellena accede a la web mediante un Código QR
2. Rellena un formulario introduciendo nombre, sabor y cantidad de bollos
3. La web envía un json por MQTT al tópico "richi5/giirrob/pr2/recibir"
4. La ESP32 recibe el pedido y lo reenvía a RoboDK por el tópico "richi5/giirrob/pr2/enviar"
5. RoboDK simula el pedido personalizado y una vez hecho devuelve "fin_produccion" al tópico "richi5/giirrob/pr2/recibir"
6. La ESP32 pondrá en marcha la cinta física de sugus para que el cliente tenga una experiencia completa

SIMULACIÓN EN ROBODK

Para la simulación de la tarea hemos empleado el software de RoboDK. Mayormente nos centraremos en explicar cómo hemos realizado de forma general la simulación, más adelante en otros apartados se podrán consultar apartados más técnicos.

Una vez terminado el programa tenemos un total de 9 scripts de python que regulan el comportamiento de todo el proceso. Se pueden agrupar en tres grupos:

- **Funcionamiento:** *ConveyorRunner*, *ConveyorRunner2*, *gestion_cajas* y *ConveyorRunner4* se encargan de gestionar las estación en general y hacer que los bollos vayan avanzando en la cinta.
- **Resets:** todos los resets recuperan inicialmente el estado de las cintas, eliminan objetos y resetean variables de la cinta necesaria
- **Otras utilidades:** *MQTT_listener* y *setter_sensores* sirven para escuchar los mensajes que mandamos por mqtt y el setter es para activar el plugin de sensores de RoboDK y que los sensores ópticos sean más manejables.

En la sección de Software comentaremos más a fondo cómo funciona el programa y cada una de sus funciones.

Hemos habilitado un [repositorio](#) en la web GitHub para que sea más fácil el acceso al código y que su lectura sea más fácil.

5. Desarrollo e Implementación

5.1. Gestión del proyecto

La gestión del proyecto la hemos realizado principalmente con el programa Trello. En él, hemos distribuido las tareas clasificándolas por fases y por etiquetas y hemos establecido fechas límite para cada una de ellas. Las cinco fases en las que las hemos distribuido son las siguientes:

1. Análisis y definición (8 - 13 abril)
 - a. Reunión inicial
 - b. Definir roles
 - c. KPIs
 - d. Diagramas de flujo
 - e. Planificación de tareas y fases
2. Equipamiento y Layout (14 - 23 abril)
 - a. Elección de robots, herramientas y elementos de la célula
 - b. Generar el layout
 - c. Distribución de elementos en la célula
 - d. Definir objetivos, targets y assets gráficos
 - e. Diseños en blender para la estación
 - f. Croquis y maqueta prototipo de la cinta
 - g. Elección y compra de materiales para la cinta física y la ESP32
3. Programación y montaje (24 abril - 20 mayo)
 - a. Python - Tratamiento de pedidos especial
 - b. Python - Empaquetado
 - c. Python - Generación de bollos
 - d. Python - Detección de bollos erróneos
 - e. Diseño página web
 - f. Contador de bollos de la web
 - g. Diseño del esquema de comunicaciones de la ESP
 - h. Comunicaciones web-ESP32
 - i. Integración de la base de datos
 - j. Montaje y creación del circuito de la cinta
 - k. Comunicaciones cinta-ESP32
4. Documentación (10 mayo - 6 junio)
 - a. Cartel expositor
 - b. Vídeo para la feria
 - c. Manual del programador
 - d. Calcular el precio total de la planta
 - e. Análisis de coste-beneficio
 - f. Diseño del Layout
 - g. Explicación del proyecto
 - h. Normativas y regulaciones en el espacio de trabajo
 - i. Tecnología, algoritmo y base de datos empleada

5. Presentación (22 - 27 mayo)
 - a. Elaboración de la presentación
 - b. Preparación del proyecto para la feria

Listado de entregables

- Business case
- Propuesta automatización
- Presupuesto y equipamiento
- Manual del proyecto
- Layout y estudio de seguridad

5.2. Pruebas y Validación

La simulación en RoboDK ha permitido validar el funcionamiento general de la estación y de los mecanismos y robots antes de su implementación física.

- Modelado del entorno: previa distribución de robots, mecanismos y demás maquinaria en la estación simulada para asegurar el correcto posicionamiento de cada elemento en la planta
- Programación de trayectorias: definición de los movimientos de los robots para evitar colisiones con los demás elementos y garantizar la seguridad con un rango óptimo
- Pruebas de coordinación: simulación de la sincronización de los robots con el resto de elementos de la estación.
- Pruebas de comunicación: simulación de la interacción de la planta con la web y la ESP32 mediante JSON a través de MQTT

Para constatar que estas comprobaciones en la simulación son efectivas en una estación real, se realizarán las siguientes pruebas de validación:

- Prueba de producción en serie: se ejecutará una tanda de 100 bolsas de bollos para ajustar la precisión y la velocidad tanto de los robots como de las cintas. El resultado esperado será una producción constante sin errores.
- Prueba de error: se evaluará la efectividad de la cámara de visión artificial y se ajustará el pistón que se encarga de desechar los bollos defectuosos. El resultado esperado será una correcta detección de la cámara y un funcionamiento delicado para el desecho del bollo por parte del pistón.
- Prueba de latencia MQTT: Se medirá el tiempo entre el envío de la petición y la ejecución de la misma en la planta. El resultado esperado será una rápida comunicación con menos de 500ms de retraso.
- Prueba de sincronización: Se verificará que los robots actúen en el momento exacto, sincronizándose con los sensores y realizando sus tareas en el momento

exacto en que el bollo llega a una determinada posición. Haremos esta prueba con 100 bollos y el resultado esperado será un total de 0 errores en el posicionamiento y la precisión de las actuaciones de los robots sobre los bollos y bolsas.

5.3. Costes y Beneficios

A continuación, adjuntamos una tabla con todos los robots, materiales, herramientas y demás maquinaria empleada en la planta robotizada. Se detallan tanto los modelos con sus características como su precio, obteniendo un precio total de la planta de 170.814€.

TIPO	MODELO	DESCRIPCIÓN	PRECIO U.	TOTAL
ROBOT COLABORATIVO (2)	UR16	carga 16kg peso 33.1kg alcance 900mm	49.800€	99.600€
CINTA TRANSPORTADORA (5)	Somefi	200mm x 250mm	3.710€	18.550€
MESA AUXILIAR (2)	Inox	1500mm x 700mm x 850mm	645€	1.290€
PISTÓN NEUMÁTICO	No hemos encontrado referencias (presupuesto estimado)		3.500€	3.500€
ESTACIÓN DE ENVASADO	Niverplast SEPP	hasta 30 bolsas/minuto	40.000€	40.000€
HERRAMIENTA INYECTOR	INOX IDM-4	dosis 40ml	465€	465€
HERRAMIENTA VENTOSAS	Robotiq EPick VAC-ES-UR-EPIC K-KIT4	1 ventosa	4.350€	4.350€
DISPENSADOR DE BOLLOS	No hemos encontrado referencias (presupuesto estimado)		2.000€	2.000€
SENSOR ÓPTICO (6)	1041452 Sick - WL12-3P2461	Sensor fotoeléctrico retrorreflectante 0-2000mm	145€	870€
CÁMARA DE DETECCIÓN DE ERRORES	Basler dart daA2500-14uc	MT9P031 Sensor 14 fps Frame rate	189€	189€
				170.814€

A estos 170.814€ de maquinaria se han de sumar los gastos de instalación, configuración y mano de obra:

- Instalación física y eléctrica: 8.000€ - 14.000€
- Programación de robots, PLC, sensores, etc.: 11.000€ - 18.000€
- Mano de obra (ingeniería y técnicos): 10.000€ - 16.000€

Lo que en total suma unos 52.000€ al presupuesto.

En cuanto al personal, su capacitación es fundamental para garantizar un buen mantenimiento de la maquinaria y un control seguro sobre la misma. Por ello, es necesario que se les imparta una buena formación:

- Interfaz hombre-máquina: formación en el control de los sistemas automatizados y aprendizaje de resolución de incidencias (1.500€ - 3.000€ por trabajador)
- Mantenimiento de los sistemas: formación en el análisis de fallos eléctricos, mecánicos o de software y mantenimiento de los robots, sensores y demás maquinaria (2.000€ - 4.000€ por trabajador)
- Seguridad en entornos robotizados: formación en normativa y seguridad en colaboración con robots y situaciones de emergencia (1.500€ - 2.000€ por trabajador)

Nuestra planta contará con dos empleados encargados de asegurar el correcto mantenimiento y el correcto uso de los sistemas, lo cual significa un gasto formativo de unos 14.000€.

Análisis coste-beneficio

Resumiendo todos los costes, la inversión inicial por el momento es de 236.814€ y estas han sido las conclusiones:

- Aumento de la productividad:
 - Reducción de errores: eliminación del 95% de los bollos detectados.
 - Producción en masa las 24 horas: antes de la automatización teníamos un tiempo de ciclo total tal que se procesaban 5 bollos/minuto con la fábrica en marcha durante 16 horas al día. Con la automatización, se consigue mejorar la producción a 23 bollos/minuto, con lo que se reduce el tiempo de ciclo en un 360%
- Ahorro en mano de obra
 - Reasignación del personal: no son necesarios los empleados que se encargaban de rellenar y empaquetar los bollos, de forma que se invertirán los 14.000€ que se han comentado anteriormente en su formación para tener los conocimientos necesarios y poder manejar y mantener la planta segura y eficientemente.

- Mayor calidad: la detección automática de errores ahorra 5.000€/año en indemnizaciones por fallos en la producción.

Retorno de la Inversión (ROI)

Los ahorros anuales estimados con este proyecto son de 655.208€. Teniendo en cuenta el salario de los dos trabajadores de mantenimiento y software de la planta, anualmente se pagarían 60.000€ en salarios, lo cual se le debe sumar a la inversión inicial y quedaría en un gasto de 296.814€.

Para ser lo más realistas posibles proponemos tres casos: positivo, realista y negativo.

En el caso positivo, la fábrica trabajaría las 24 horas del día todos los días por la altísima demanda del producto y se venderían todos y cada uno de los bollos que se producen. El beneficio anual (teniendo en cuenta que por el aumento de producción se sumarían 1.033.680€) sería de :

$$\text{ROI: } \frac{(1.033.680 + 5.000) - 296.814}{296.814} * 100 = 249\%$$

De esta forma, tardaríamos entre 4 y 5 meses en recuperar la inversión.

En el caso promedio, la fábrica se pondría en marcha durante 12 horas cada día, ya que en un caso realista no se vendería toda la producción de un día entero. Por tanto, el beneficio anual (sumando la producción de las 12 horas de 429.240€ anuales) sería de:

$$\text{ROI: } \frac{(429.240 + 5.000) - 296.814}{296.814} * 100 = 46\%$$

De esta forma, tardaríamos unos 2 años y 2 meses en recuperar la inversión.

Por último consideramos el caso negativo, en el que la fábrica solo estaría en marcha 8 horas al día debido a la baja demanda de bollos. Por tanto, la producción de 8 horas al día sumaría 227.760€ anuales:

$$\text{ROI: } \frac{(227.760 + 5.000) - 296.814}{296.814} * 100 = -21\%$$

Por tanto, en el caso de que hubiera tan baja demanda no sería viable automatizar la planta.



6. Normativa y Regulación. Seguridad

La automatización de la planta BollyKO's, que integra robots tipo cobots y sistemas embebidos, debe cumplir con la normativa vigente tanto española como europea, especialmente en materia de **seguridad laboral**¹ y **regulación técnica de maquinaria**.

Seguridad laboral

La base legal en España es la **Ley 31/1995** de Prevención de Riesgos Laborales, desarrollada por el **Real Decreto 1215/1997**, que obliga a garantizar condiciones óptimas y seguras para los trabajadores que utilizan equipos de trabajo automatizados. Esto implica proteger frente a elementos móviles, instalar dispositivos de parada de emergencia y formar al personal de forma adecuada.

En nuestro caso, el uso de robots en el ámbito industrial exige barreras de seguridad, sensores fotoeléctricos y un botón de emergencia funcional, ya integrado tanto en nuestro layout físico como en la simulación en RoboDK. Aunque **la normativa permite el uso de cobots sin vallado físico si se cumplen ciertos requisitos técnicos**, en nuestro proyecto hemos optado por aplicar **medidas adicionales de seguridad** para asegurar que no existe posibilidad de error ni riesgo para los operarios. También se han previsto formaciones específicas para el manejo de los sistemas automatizados y los riesgos asociados.

Regulación técnica y marcado CE

Todos los componentes industriales, incluidos los robots, deben cumplir la **Directiva 2006/42/CE** (Directiva de Máquinas), transpuesta al derecho español mediante el **RD 1644/2008**. Esto implica que todo el sistema debe disponer del **marcado CE**, tras una evaluación de riesgos conforme a la **ISO 12100**, asegurando que cumple con los requisitos esenciales de seguridad (resguardos, sistemas de parada, control fiable...).

Con vistas a futuro, cabe tener en cuenta el nuevo **Reglamento (UE) 2023/1230**, que será obligatorio a partir de 2027 y reforzará aspectos como la robótica colaborativa. En nuestro caso, este nuevo marco permitiría incluso ser más flexibles en ciertas medidas de seguridad sin comprometer la integridad del sistema, lo que a su vez puede reducir el coste de implementación.

¹ Normativa referenciada en la sección 9.-“Referencias”

Principales normativas y estándares aplicables

- **Ley 31/1995** de Prevención de Riesgos Laborales
- **Real Decreto 1215/1997**, sobre la utilización de equipos de trabajo
- **Directiva 2006/42/CE** sobre seguridad de las máquinas
- **Real Decreto 1644/2008**, sobre comercialización de maquinaria
- **Reglamento (UE) 2023/1230**, nueva regulación de máquinas (aplicable a partir de 2027)
- **UNE-EN ISO 10218-1:2012** e **ISO 10218-2:2011**, sobre seguridad en robots
- **ISO/TS 15066:2016**, específica para robots colaborativos
- **ISO 12100:2010**, principios generales de diseño seguro de máquinas
EN ISO 13849-1:2016 / **IEC 62061:2021**, sobre seguridad funcional en sistemas de control.

Aplicación práctica de la normativa en BollyKO's

Durante el desarrollo del proyecto BollyKO's se han aplicado medidas concretas para cumplir con la normativa de seguridad laboral y técnica:

- **Seguridad laboral:** Se ha incorporado un botón físico de parada de emergencia conectado a la ESP32 mediante interrupción. Además, en RoboDK se han simulado condiciones seguras, y se ha considerado la formación del personal como un elemento esencial.
- **Regulación técnica:** Todos los componentes seleccionados cuentan con marcado CE. Se han aplicado soluciones basadas en ISO 12100, como el uso de sensores, visión artificial y lógica de control distribuida para minimizar riesgos.
- **Robótica colaborativa:** Aunque el uso de cobots permite operar sin vallados físicos según ISO/TS 15066, en BollyKO's hemos reforzado las medidas de seguridad con control redundante y sistemas de detección adicionales, anticipándonos al nuevo reglamento europeo de máquinas.
- **Seguridad funcional:** El control bajo FreeRTOS organiza el sistema en tareas concurrentes protegidas con mutex y semáforos, garantizando fiabilidad ante eventos críticos.

Conclusión

La implementación de automatización avanzada debe regirse por un doble eje normativo: la legislación de seguridad laboral y la regulación técnica de seguridad de máquinas. No basta con que un sistema tenga marcado CE; debe integrarse correctamente, con formación adecuada y condiciones de operación seguras. En BollyKO's hemos cumplido con todos estos requisitos, implementando incluso medidas adicionales para garantizar que el sistema es totalmente seguro y listo para ser implantado en un entorno industrial real.

7. Desarrollo de Software y Algoritmos

MODULO ESP32

Listado tecnológico

ELEMENTO	DETALLES
Placa controladora	ESP32-s3
Lenguaje de programación	C empleando Arduino y FreeRTOS
Comunicaciones	Wifi (conexión al broker MQTT) y HTTP (envío a API externa en Render)
Protocolos	MQTT (mensajería entre web, ESP32 y RoboDK) y HTTP POST (registro defectuosos)
Bibliotecas utilizadas	ArduinoJson, Bounce2, ESP32Servo, EventButton y PubSubClient.
Entorno de desarrollo	Arduino
Broker MQTT	Broker público broker.emqx.io
Servidor externo	Render (para la base de datos)

Descripción de la implementación

Estructura general

El código se divide en múltiples módulos:

- config.h
- ESP32-S3-IoT-Device.ino
- c_logger.ino
- d_wifi_lib_no_tocar.ino
- e_mqtt_lib_no_tocar.ino
- f_funciones.ino
- g_comunicaciones.ino
- s_setup.ino
- w_loop.ino
- basic_Task_QR.ino

Componentes y tareas principales

COMPONENTE	DESCRIPCIÓN
Recepción de pedidos	Se parsean mensajes JSON entrantes y se añaden a una cola circular como tareas pendientes.
Procesador de tareas	Tarea que saca de la cola y actualiza las estructuras pedidoActual y ColaPedidos.
Control dispensador	Espera señal de fin de producción y activa el servo correspondiente y la cinta transportadora.
Pulsador de emergencia	Interrupción hardware: alterna eventos “start”/”stop” via MQTT según se presione el botón.
Cámara y QR	Captura imágenes con la cámara de la ESP, decodificar QR con quirc y actualizar la base de datos vía MQTT.

Algoritmos y estructuras avanzadas

Multitarea con FreeRTOS

- Cada proceso principal (lectura QR, recepción, ejecución de pedidos, control de servos, botón) es una tarea separada
- Uso de `xTaskCreatePinnedToCore()` que permite decidir el núcleo de ejecución.

Colas circulares personalizadas

Se implementa una estructura de cola circular doble protegida por `SemaphoreHandle_t` mutex:

```
struct ColaPedidos {
    String* sabores;
    int* cantidades;

    int* frenteS;
    int* finS;
    int* tamañoS;

    int* frenteC;
    int* finC;
    int* tamañoC;

    SemaphoreHandle_t mutex;
};
```

Permite gestionar dos flujos independientes (sabores y cantidades) sincronizados, pero con posibilidad de crecer o vaciarse de forma independiente. Se protege su acceso en regiones críticas con xSemaphoreTake() y xSemaphoreGive().

Colas circulares personalizadas

Se usa xQueueCreate() para mantener las tareas del sistema, las tareas están compuestas por una estructura:

```
typedef struct {  
    String tipo;  
    String nombre;  
    String sabor;  
    int cantidad;  
} Tarea;
```

Permite expansión futura para añadir distintos tipos de tareas como paradas programadas.

Procesamiento de QR en tiempo real

- La tarea QRCodeReader usa la librería quirc sobre imágenes capturadas con esp_camera.
- Si el QR es legible, se lanza un POST HTTP con el hecho_id correspondiente al backend de Render.

Lógica de control de I/O

- Cada servo está encapsulado en funciones tipo correspondientes a cada sabor: activarServoChocolate(cant).
- Se controla un único motor de cinta, independientemente del sabor pedido
- El botón de parada genera eventos controlados a través de interrupciones

ROBODK

En esta sección iremos desglosando el funcionamiento de los distintos archivos que regulan el comportamiento de RoboDK uno a uno de los programas que hacen funcionar la cinta y que necesitan explicación. Además comentaremos ciertas decisiones de diseño que hemos tenido tras hacer las pruebas detalladas en anteriores apartados.

- *ConveyorRunner*: Este es uno de los programas más extensos de la implementación. Consta de dos hilos, uno de ellos se encarga de gestionar la aparición de bandejas (“trays”) tanto las normales de producción en masa como las de pedidos (“color personalizado”). El hilo principal se encarga de mover o parar la cinta, ejecutar el relleno e ir eliminando los bollos que lleguen al final de la cinta, además de manejar los pedidos y los eventos como la parada de emergencia o el bollo rechazado. Los hilos se ayudan de una serie de funciones auxiliares que le ayudan a hacer ciertas comprobaciones y acciones
- Antes de continuar, cabe explicar cómo hemos gestionado los pedidos. Cuando el `mqtt_listener` recibe un pedido pone en robodk una variable del estilo: “pedido_<nombre>_<sabor>”, cuando la primera cinta lo lee, crea una variable llamada `seguro` para guardar la cantidad de bollos que se han pedido y se van generando bollos hasta que la variable “pedido...” es igual a 0, lo que significa que sus bollos ya se están rellenando en la planta. Cuando el sensor del final de la cinta detecta que la bolsa a borrar es una de un pedido (por su nombre), abre una variable llamada “rellenados_<nombre><sabor>” y cuando esta variable es igual a “seguro_<nombre>_<sabor>” sabemos que el pedido ha sido completado
- *ConveyorRunner2*: este script maneja la segunda cinta y genera tantas bolsas se hayan rellenado y gestionado en la primera (variable `empaquetados_normal`) asegurando así datos realistas y ajustados a una fábrica. Avanza las bolsas hasta que llegan a un sensor donde el UR 16e realiza el pick and place de esta bolsa en las cajas. Además las va dejando una encima de otra, para que visualmente parezca que se van apilando en la caja.
- *Gestión_cajas*: maneja la cinta que hace que cuando hay tres bolsas en una caja, la caja se vaya (al llegar a un sensor se elimina la caja y las bolsas) y aparezca una nueva.
- *ConveyorRunner4*: esta cinta gestiona la cinta de los pedidos, primero espera a que la variable “seguro_[...]” sea igual a la variable “pedido_[...]”. Una vez iguales, comienza a extraer el valor del pedido de la persona y el sabor. Genera el pedido con la bolsa personalizada correspondiente y manda un MQTT de fin de producción para que la esp habilite la cinta transportador física

Inicialmente se utilizó una implementación basada completamente en bucles `while(True)`, pero este enfoque resultó ineficiente debido a las esperas activas y al exceso de comprobaciones, lo que dificultaba la legibilidad y afectaba el rendimiento. También se probó un enfoque multihilo completo, pero tampoco ofrecía mejoras significativas. Por ello, se optó por una solución mixta, que combina hilos solo donde son realmente necesarios. Esta arquitectura consigue generar un pedido completo en aproximadamente un minuto, un tiempo razonable dado el nivel de sincronización y carga computacional.

Además, se ha implementado un mecanismo de exclusión mutua mediante semáforos para gestionar correctamente la copia y creación de objetos en RoboDK. Esta medida es esencial, ya que la API de RoboDK no tolera múltiples accesos concurrentes a las funciones de duplicación, lo que provocaba la aparición de objetos en cintas incorrectas. Aunque introduce cierta latencia, este enfoque garantiza consistencia y estabilidad en la simulación².

PÁGINA WEB

Listado tecnológico

ELEMENTO	DETALLES
Fronted	HTML, JavaScript, CSS
Lenguaje de programación	Python
Comunicaciones	HTTP (formularios y BBDD (la visualización en la web)) y WebSocket MQTT
Protocolos	MQTT (envío de pedidos) y HTTP POST (registro defectuosos)
Librerías	Paho MQTT, API para POST, os (para más seguridad), Json
Entorno de desarrollo	Github
Broker MQTT	Broker publico broker.emqx.io (En wss con puerto 8084 para más seguridad)
Servidor	Render (servicio web y Base de Datos)

Descripción de la implementación

- El usuario accede a la web a través del código QR
- Puede hacer un pedido personalizado rellenando el formulario (nombre, sabor, cantidad)

² Aún siguen saliendo ciertos fallos, pero hablamos de momentos anecdóticos y debido a limitaciones del propio RoboDK al gestionar tantos objetos de forma seguida copiándolos y pegándolos y eliminándolos de forma tan continua.

No pierdas la oportunidad de probar los bolliKaos 🍩

Formulario de pedido

Nombre:

Sabor:

Número de bollos (1-3):

Prueba de calidad: genera un bollo defectuoso ▼

- Al enviar el formulario
 - Se envía un mensaje MQTT con los datos del pedido en formato json
 - Se guarda el pedido en la base de datos (Tabla Pedidos y Hechos)
- Los contadores de producción (por sabor y totales) se actualizan dinámicamente desde /estado (Controlado desde el appy) y se muestra en la página web (La foto muestra los pedidos que tuvimos en la feria ya que nuestra base de datos registro la información de cada cliente)



- Existe una sección con código QR para generar un producto defectuoso a través de la lectura de la cámara de la ESP32.
- Hay secciones extra accesibles desde el menú:

- Nosotros: Muestra un poco de información sobre nosotros y nuestras fotos
- Planta de producción: Se muestra nuestra planta de robodk desde distintas perspectivas, además de nuestro layout.

(Estas secciones extra fueron creadas para la feria, de esta manera si alguien tenía más curiosidad se podían meter).

Estructura general

El código se divide en múltiples módulos:

- index.html : Página principal con el formulario de pedidos y la visualización de la base de datos
- script.js: Lógica json para enviar formularios, gestionar mqtt y actualizamos los contadores (donde se visualiza la base de datos)
- style.css: Los estilos utilizados para cada sección del código
- app.py: Backend Flask con rutas para paginas, envio de pedidos a la BBDD, datos del estado y defectuosos
- nosotros.html: Página sobre el equipo
- planta.html: Página sobre el sistema automatizado
- Carpeta foto: Contiene las imágenes utilizadas

Componentes

COMPONENTE	DESCRIPCIÓN
Formulario	Se rellenarán los datos pedidos (nombre, sabor, cantidad) y una vez le demos a enviar, se enviará un mensaje MQTT
Contadores (Visualización BBDD)	Se visualizará la base de datos a través de /estado, gestionado por el app.py
Código QR	Tendrá un desplegable para visualizar el código QR para los defectuosos (y si queremos probarlo, la camara esp32 lo procesará)
Backend	Tendrá todas las rutas necesarias (entre ellas /pedido, /estado, /api/defectuoso) para actualizar la base de datos

Consultas de la base de datos

Para realizar las consultas en la base de datos, lo hacemos en el archivo [app.py](#). Para cada uno, se ha utilizado una ruta diferente

1. Consulta del estado global

Se llama tras enviar un pedido o pasados 10 segundos para actualizar los contadores de pedidos, hechos y defectuosos en la web. Se utiliza la ruta /estado.

```
query_db("SELECT sabor, COUNT(*) FROM PEDIDOS GROUP BY sabor;")
query_db("SELECT sabor, COUNT(*) FROM HECHOS GROUP BY sabor;")
query_db("SELECT COUNT(*) FROM HECHOS;", one=True)[0]
query_db("SELECT SUM(cantidad) FROM DEFECTUOSOS;", one=True)
```

2. Inserción de nuevo pedido

Cuando un cliente envía el formulario de pedido, guardamos sus datos para registrarlos en la BBDD y simular su fabricación. Se utiliza la ruta /pedido.

```
cur.execute(
    "INSERT INTO PEDIDOS (nombre, sabor, cantidad,
    fecha) VALUES (%s, %s, %s, %s) RETURNING id;",
    (nombre, sabor, cantidad, fecha_hoy)
)
```

E inmediatamente se insertarán los bollos hechos simulados

```
for _ in range(cantidad):
    cur.execute(
        "INSERT INTO HECHOS (pedido_id, sabor, fecha_hecho) VALUES
        (%s, %s, %s);",
        (pedido_id, sabor, fecha_hoy)
    )
```

3. Inserción de bollo defectuoso

Cuando se detecta un bollo defectuoso, los registra permitiendo un seguimiento del sistema más realista. Se utiliza la ruta /api/defectuoso

```
cur.execute(
    "INSERT INTO DEFECTUOSOS (hecho_id, cantidad, motivo) VALUES
    (%s, %s, %s);",
    (hecho_id, 1, motivo)
)
```



BASE DE DATOS

Listado tecnológico

ELEMENTO	DETALLES
Servidor	Render
Lenguaje	SQL
Comunicaciones	Python (La web (con un fichero app.py) y la esp32 (enviaba un HTTP a la appy de la web), con un flask se actualizaba la base de datos.
Protocolos	Web (registro hechos y pedidos) y Esp32 (registro defectuosos)
Entorno de desarrollo	PgAdmin4

Esquema de la base de datos

La Base de datos implementada utiliza PostgreSQL y se aloja en el servicio Render. Está conectada a la página web para que nuestros clientes puedan ver en todo momento nuestras estadísticas. Se actualizará mediante el formulario de la página web con los datos registrados (tabla pedidos y hechos), además de la ESP32 (tabla defectuosos). Este es el esquema completo definido por el lenguaje SQL.

La tabla de pedidos, registra cada pedido desde el formulario de la web. Formada por:

- ID: Cada pedido tiene un ID para identificarlo.
- Nombre: En el formulario pedimos el nombre, para que el id de robodk sea el nombre con números y así el usuario tenga una experiencia más personalizada.
- Sabor: Tenemos tres sabores: chocolate, fresa y vainilla.
- Cantidad: Esto registrará la cantidad de pedidos que hemos tenido en la feria
- Fecha: Con esto llevaremos un control más exacto de la hora a la que se ha pedido

```
CREATE TABLE PEDIDOS (  
    id SERIAL PRIMARY KEY,  
    nombre TEXT NOT NULL,  
    sabor TEXT NOT NULL,  
    cantidad INTEGER NOT NULL,  
    fecha DATE  
);
```

	id [PK] integer	nombre text	sabor text	cantidad integer	fecha date
63	63	Ruben	vainilla	3	2025-05-30
64	64	Marta	fresa	1	2025-05-30
65	65	Ivan	vainilla	2	2025-05-30
66	66	Marcos	fresa	1	2025-05-30
67	67	Lucia	fresa	3	2025-05-30
68	68	Daniel	chocolate	2	2025-05-30
69	69	Laura	vainilla	1	2025-05-30
70	70	Carlos	fresa	3	2025-05-30
71	71	Silvia	chocolate	3	2025-05-30

La tabla de hechos, es un contador de bollos para registrar cuántos llevamos hechos de cada sabor. Formada por:

- ID: Cada pedido tiene un ID para identificarlo.
- pedido_id: Necesario para poder unir la tabla de pedidos con hechos y que tengan el mismo identificador
- sabor: Para llevar un control de los bollos por sabores (chocolate, fresa y vainilla).
- fecha_hecho: Con esto llevaremos un control más exacto de la hora a la que se ha hecho

```
CREATE TABLE HECHOS (
    id SERIAL PRIMARY KEY,
    pedido_id INTEGER NOT NULL,
    sabor TEXT NOT NULL,
    fecha_hecho DATE,
    FOREIGN KEY (pedido_id) REFERENCES PEDIDOS(id)
);
```

	id [PK] integer	pedido_id integer	sabor text	fecha_hecho date
136	136	67	fresa	2025-05-30
137	137	68	chocolate	2025-05-30
138	138	68	chocolate	2025-05-30
139	139	69	fresa	2025-05-30
140	140	69	fresa	2025-05-30
141	141	70	vainilla	2025-05-30
142	142	71	chocolate	2025-05-30
143	143	71	chocolate	2025-05-30
144	144	71	chocolate	2025-05-30

La tabla de defectuosos, es un contador de bollos defectuosos. Esta tabla se controla mediante la esp32, a partir de la lectura de un código Qr que está dentro de la web. Formada por:

- ID: Cada pedido tiene un ID para identificarlo.
- hecho_id: Necesario para poder unir la tabla de hechos con defectuosos y que tengan el mismo identificador
- cantidad: Necesitamos llevar un control de cuantos bollos defectuosos hemos tenido.
- motivo: Cada bollo defectuoso necesita tener una explicación de por qué está defectuoso.

```
CREATE TABLE DEFECTUOSOS (
    id SERIAL PRIMARY KEY,
    hecho_id INTEGER NOT NULL,
    cantidad INTEGER NOT NULL,
    motivo TEXT,
    FOREIGN KEY (hecho_id) REFERENCES HECHOS(id)
);
```

	id [PK] integer	hecho_id integer	cantidad integer	motivo text
1	1	45	1	bordes quemados
2	2	65	1	submezcla incorrecta
3	3	81	1	relleno insuficiente
4	4	136	1	exceso de cocción
5	5	27	1	fallo de molde
6	6	98	1	forma irregular
7	7	5	1	relleno insuficiente
8	8	32	1	fallo de molde
9	9	42	1	rotura en cinta

Los ejemplos utilizados son los datos que obtuvimos en la feria por algunos de nuestros clientes

8. Conclusiones y recomendaciones

Este proyecto fue concebido, no sólo para cumplir los objetivos industriales de mejorar los KPI's, mejorar rendimiento, incrementar beneficios... También ha estado influenciado por dar una buena impresión en la Feria de Proyectos y que fuera atractivo para el público.

Al final hemos conseguido obtener una producción en masa que no cesa con estándares industriales junto a una parte de interacción que nos ha servido para ser atractivos en la Feria. Todo esto mejorando los KPI's propuestos de forma realista y aportando una solución viable y sencilla a un problema plausible.

No solo hemos tenido que emplear los conocimientos de PR2 o las asignaturas de este cuatrimestre (colas, programación en Arduino, RoboDK) sino que también hemos empleado conocimientos de asignaturas pasadas y hemos adquirido conocimientos en campos que no hemos tocado para nada (semáforos, hilos, comunicaciones, diseño web, diseño y montaje de prototipo 3D, Blender...).

Todos estos desafíos han sido superados incluso con el poco tiempo disponible y la gran carga de trabajo general a la que nos enfrentamos.

En resumen, no solo hemos aprendido conocimientos técnicos, sino que también hemos sabido manejar el tiempo de forma eficiente, trabajar los 4 miembros del grupo en paralelo y adelantar trabajo e incluso adelantar plazos de entrega. Planificación, orden, interés y mucho esfuerzo que han culminado en un proyecto del que estamos muy orgullosos.

Recomendamos que el proyecto se considere como una solución industrial, y que se valore el coste de añadirle una parte de personalización e interacción sin cesar en la producción en masa. Y también valorar que no solo hemos hecho desarrollo de software, sino que es un proyecto multidisciplinar donde hemos trabajado desde material, montaje de prototipos y planos, hasta el desarrollo puramente de software, pasando por backend, frontend y diseños de BBDD.

9. Referencias

Normativa consultada para la sección 6

- BOE. (1995). *Ley 31/1995, de 8 de noviembre, de Prevención de Riesgos Laborales*. Boletín Oficial del Estado, nº 269, pp. 32590-32611. <https://www.boe.es/eli/es/l/1995/11/08/31>
- BOE. (1997). *Real Decreto 1215/1997, de 18 de julio, por el que se establecen las disposiciones mínimas de seguridad y salud para la utilización por los trabajadores de los equipos de trabajo*. Boletín Oficial del Estado, nº 188, pp. 24324-24332. <https://www.boe.es/eli/es/rd/1997/07/18/1215>
- BOE. (2008). *Real Decreto 1644/2008, de 10 de octubre, por el que se establecen las normas para la comercialización y puesta en servicio de las máquinas*. Boletín Oficial del Estado, nº 254, pp. 41935-41956. <https://www.boe.es/eli/es/rd/2008/10/10/1644>
- Parlamento Europeo y Consejo de la Unión Europea. (2006). *Directiva 2006/42/CE del Parlamento Europeo y del Consejo, de 17 de mayo de 2006, relativa a las máquinas y por la que se modifica la Directiva 95/16/CE*. Diario Oficial de la Unión Europea, L 157, 9–48. <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX%3A32006L0042>
- Parlamento Europeo y Consejo de la Unión Europea. (2023). *Reglamento (UE) 2023/1230 del Parlamento Europeo y del Consejo, de 14 de junio de 2023, relativo a las máquinas y por el que se deroga la Directiva 2006/42/CE*. Diario Oficial de la Unión Europea, L 165, 1–99. <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX%3A32023R1230>
- ISO. (2012). *UNE-EN ISO 10218-1:2012: Robots y dispositivos robóticos – Requisitos de seguridad para robots industriales – Parte 1: Robots*. Organización Internacional de Normalización.
- ISO. (2011). *UNE-EN ISO 10218-2:2011: Robots y dispositivos robóticos – Requisitos de seguridad para robots industriales – Parte 2: Integración de sistemas de robot*. Organización Internacional de Normalización.
- ISO. (2016). *ISO/TS 15066:2016: Robots y dispositivos robóticos – Robots colaborativos (Especificación técnica)*. Organización Internacional de Normalización.
- ISO. (2010). *ISO 12100:2010: Seguridad de las máquinas – Principios generales para el diseño – Evaluación y reducción del riesgo*. Organización

Internacional de Normalización.

- ISO. (2016). *EN ISO 13849-1:2016: Seguridad de las máquinas – Partes de los sistemas de control relativas a la seguridad – Parte 1: Principios generales para el diseño*. Organización Internacional de Normalización.
- IEC. (2021). *IEC 62061:2021: Seguridad funcional de los sistemas eléctricos, electrónicos y electrónicos programables relacionados con la seguridad*. Comisión Electrotécnica Internacional.

10. Anexos

10.1. Manual de Instalación y Funcionamiento

- **Requisitos previos:**
 - Conexión a internet estable
 - Acceso a RoboDK con los scripts y la estación
 - Navegador compatible con HTML5 y WebSocket
 - Placa ESP32-S3 (módulo de cámara) flasheada con el código proporcionado
 - Acceso al servidor en Render y a la BBDD de PostgreSQL
- **Instalación:**
 - Instalar Arduino IDE y configurar la ESP32.
 - Instalar bibliotecas: PubSubClient, ESP32Servo...
 - Cargar el código
 - Conectar físicamente los componentes(esquemas mostrados anteriormente).
- **Configuración RoboDK:**
 - Abrir archivo .rdk
 - Ejecutar el script “SETTER SENSORES” ubicada en la carpeta main
 - Ejecutar el MAIN
 - Comprobar el correcto funcionamiento durante 30s antes de enviarle cualquier petición
 - En caso de bugs, pulsar esc repetidamente y volver a ejecutar el MAIN (la estación se reiniciará sola).
- **Configuración de la página web:**
 - Acceder a la URL pública: <https://richibollos.onrender.com>, o escanear el QR.
 - Una vez en la web, esperar 5s a que la base de datos se actualice y muestre correctamente el estado de producción
 - Hacer un pedido y comprobar que se envía, que lo recibe la ESP y que se lo manda a RoboDK de forma correcta

- **Funcionamiento general**

- Usuario hace un pedido por la web
- Backend lo guarda en la BBDD y lanza el pedido por MQTT
- RoboDK simula el pedido, lo procesa y envía un *fin_produccion*.
- ESP activa la cinta y dispensador correspondiente según sabor y cantidad
- La cámara simula si hay bollos defectuosos (QR en la web): si hay defecto, se elimina del RoboDK y se envía a la BBDD
- El botón de parada esta atento a si es pulsado para parar la estación.
- Todo esto sin cesar la producción en masa del RoboDK

10.2. Manual del programador

En esta sección vamos a detallar cómo usar el archivo.rdk para su correcto funcionamiento.

- En primer lugar habría que abrir la carpeta llamada MAIN y ejecutar el programa “SETTER SENSORES”. Este programa abrirá una lista con los sensores de la cinta 5 veces, hay que ir clicando uno por uno para que se vayan activando. Este paso es importante y sin él no funcionará nada.
- Una vez hecho podemos pulsar el “MAIN”, este ejecutará todos los resets para preparar la estación. Seguidamente ejecutará todos los programas .py para que la estación funcione a pleno rendimiento(producción en masa, producción personalizada, parada emergencia, simulación bollo defectuoso).
- Si queremos detener la simulación, bastará con pulsar la tecla esc varias veces y RoboDK saldrá de los programas. Si la queremos volver a ejecutar podemos pulsar el main de nuevo ya que incluye los resets necesarios para que funcione correctamente
- Si quiere ver los programas o activar alguno, puede abrir las carpetas llamadas “programa”, donde encontrará los programas individuales que afectan a esa estación.
- Si quiere ver la distribución de los objetos y los frames, abra las carpetas de “Estación” para poder ver los objetivos, los frames y objetos de referencia, cintas, robots...
- Recomendamos ver cómo se generan los objetos y van cambiando sus nombres, apareciendo y desapareciendo en los frames dentro de las cintas.
- También recomendamos encarecidamente no cambiar nada ni de lugar en la estación ni en el árbol de objetos, ni de nombre, pues esto haría que se rompiera el correcto funcionamiento del código y haría imposible la ejecución de los códigos.

10.3. Otra documentación adicional

- **Especificaciones técnicas de la cinta física:**
 - Diseño CAD realizado con FreeCAD. Archivos .stl disponibles bajo petición
 - Impresora: Wanhao Duplicator 6, PLA 1.75mm
 - Velocidad calculada con motorreductor de 100 rpm y poleas de 20 dientes.
 - Controlado con relé Keyes SR1 y ESP32
- **Enlaces de interés**
 - Repositorio del proyecto de RoboDK: [GitHub del proyecto](#)
- **Referencias a recursos de IA empleados:**
 - Chat GPT fue usado para revisar la redacción de este documento y estimar ciertos costes relativos al presupuesto del que carecemos de información.
 - Las respuestas siempre fueron revisadas por el equipo y no se han realizado copias de las respuestas.